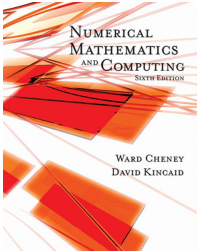
Floating-Point Representation and Error Analysis

Chapter 1 of Numerical Analysis

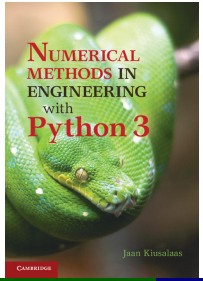
Yiping Cheng
ypcheng@bjtu.edu.cn
29-Jul-2022

School of Electronic and Information Engineering
Beijing Jiaotong University

Course Syllabus



- **Textbook:** Numerical Mathematics and Computing (6th Edition), Ward Cheney and David Kincaid, Thomson
- **Materials covered:** Chapters 1–11 of the textbook
- **Composition of final grade:** Assignments (20%), attendance (10%), final examination (70%)



Chapter Contents

1 Floating-Point Representation
- Floating-Point Formats
- Rounding
- Operations

2 Error Analysis
- Some Concepts for Error Analysis
- Error Analysis for Basic Operations
- Some Error-Avoiding Techniques
- Review of Taylor's Formula

Floating-Point Representation

Number representations: fixed-point and floating-point

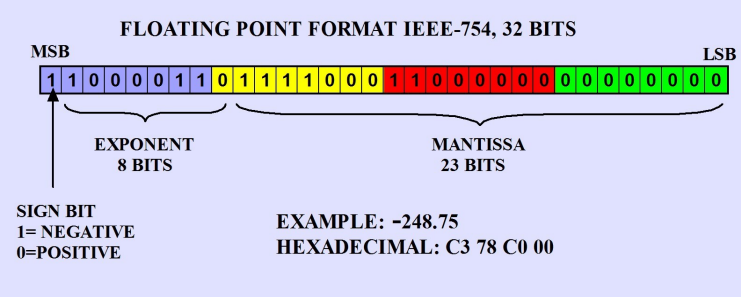
C Type	#Bits	Fixed or Floating
char	8	fixed
short	16	fixed
int	32	fixed
long long	64	fixed
float	32	floating
double	64	floating

Fixed-point representation is for integers, and floating-point representation is for real numbers.

Comparison between fixed and floating-point representations

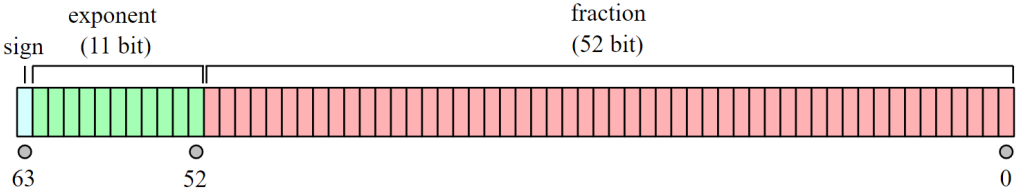
Fixed-Point	**Floating-Point**
evenly distributed	unevenly distributed
can't represent fractions	can represent fractions
small range	big range
exact computation	inexact computation
faster computation	slower computation

IEEE 754 single-precision floating-point format



1) If $e = 255$ and $f \neq 0$, then v is NaN regardless of s
2) If $e = 255$ and $f = 0$, then $v = (-1)^s \infty$
3) If $0 < e < 255$, then $v = (-1)^s 2^{e-127}(1 \cdot f)$
4) If $e = 0$ and $f \neq 0$, then $v = (-1)^s 2^{-126}(0 \cdot f)$ (denormalized numbers)
5) If $e = 0$ and $f = 0$, then $v = (-1)^s 0$ (zero)

IEEE 754 double precision floating-point format



1) If $e = 2047$ and $f \neq 0$, then v is NaN regardless of s
2) If $e = 2047$ and $f = 0$, then $v = (-1)^s \infty$
3) If $0 < e < 2047$, then $v = (-1)^s 2^{e-1023}(1 \cdot f)$
4) If $e = 0$ and $f \neq 0$, then $v = (-1)^s 2^{-1022}(0 \cdot f)$ (denormalized numbers)
5) If $e = 0$ and $f = 0$, then $v = (-1)^s\ 0$ (zero)

Things remarkable

1. There exist ∞ and $-\infty$.
2. There exist NaNs (Not-a-Numbers).
3. There exist denormalized (subnormal) numbers. Unlike normalized numbers, with denormalized numbers we do not assume the leading digit 1 in the mantissa. It is quite rare that we encounter denormalized numbers but they are nonetheless needed for the completeness of the floating-point system. In essence, they allow gradual underflow, i.e. a smooth transition from normalized numbers to zero.
4. There exist two zeros. But they should be treated as the same zero.

Rounding

Rounding takes a number regarded as infinitely precise and, if necessary, modifies it to fit in the destination's format while signaling the inexact exception, underflow, or overflow when appropriate.

Except where stated otherwise, every operation shall be performed as if it first produced an intermediate result correct to infinite precision and with unbounded range, and then rounded that result according to one of the rounding methods.

There are many rounding methods. Here we only describe the default method used in floating-point computations: **roundTiesToEven**. And we will assume throughout this course that this method is used.

roundTiesToEven: choose nearest, when two tie, choose even

The floating-point number nearest to the infinitely precise result shall be delivered; if the two nearest floating-point numbers bracketing an unrepresentable infinitely precise result are equally near, the one with an even least significant digit shall be delivered.
When near infinity,

- Single-precision: The biggest finite representable number is $f_{max} = 2^{127} * 1.11111111111111111111111$ $2^{127} * (2 - 2^{-23})$. Any number $\geq 2^{128}$ should be represented as ∞. Moreover, according to roundTiesToEven, any number equal to or bigger than

$$\frac{2^{127} * 1.11111111111111111111111b + 2^{128}}{2} = 2^{127} * (2 - 2^{-24})$$

  should be represented as ∞.

- Double precision: The situation is similar, omitted.

Illustrating roundTiesToEven

The exact binary fractional expansion of $\frac{2}{3}$ is 0.10101010101010101010101010101010..., which should be (allowing infinite presision) represented as $2^{-1} * 1.01010101010101010101010\ 101010....$ Now since there are only 23 bits of mantissa, its single-precision floating-point representation, using roundTiesToEven, is $2^{-1} * 1.01010101010101010101011$.

The exact binary fractional expansion of $\frac{22369621}{2^{25}}$ is 0.10101010101010101010101010101, which should be represented as $2^{-1} * 1.01010101010101010101010\ 1$. Now since there are only 23 bits of mantissa in single-precision floating-point, the last bit does not fit into the representation. This is a tie case, in this case we let the least significant bit be even, which is 0, therefore this last bit is discarded, and its single-precision representation is $2^{-1} * 1.01010101010101010101010$.

In fact, $\frac{22369621}{2^{25}}$ and $\frac{22369620}{2^{25}}$ have the same single-precision floating-point representation.

Operations

IEEE 754 standard lists 5 basic operations: **addition**, **subtraction**, **multiplication**, **division**, and **square root**. Each of these operations must be performed as if it first produced an intermediate result correct to infinite precision and with unbounded range, and then rounded that intermediate result, if necessary, to fit in the destination's format. In other words, the basic operations are required to be completely correct-rounded.

There is of course no problem for comparison operations to be fully exact.

Elementary functions such as exp, log, sin, arctan, etc., though the more accurate the better, are not required by the standard to be completely correct-rounded.

Some notation

Let us establish the following notation:

Symbol	Meaning
$+0$	the floating-point zero with sign bit=0
-0	the floating-point zero with sign bit=1
0	either $+0$ or -0
$+a$	any positive finite floating-point number
$-a$	any negative finite floating-point number
a	any nonzero finite floating-point number
$+\infty$	the positive infinity
$-\infty$	the negative infinity
∞	either $+\infty$ or $-\infty$
NaN	any Not-a-Number

When is the result infinite?

Cases when result=$+\infty$	**Cases when result=$-\infty$**
positive overflow[1]	negative overflow[2]
$+\infty + +\infty, +\infty + a, +\infty + 0$	$-\infty + -\infty, -\infty + a, -\infty + 0$
$+\infty - -\infty, +\infty - a, +\infty - 0$	$-\infty - +\infty, -\infty - a, -\infty - 0$
$a - -\infty, 0 - -\infty$	$a - +\infty, 0 - +\infty$
$+\infty * +\infty, -\infty * -\infty$	$+\infty * -\infty$
$+\infty * +a, -\infty * -a$	$+\infty * -a, -\infty * +a$
$\frac{+\infty}{+a}, \frac{-\infty}{-a}$	$\frac{+\infty}{-a}, \frac{-\infty}{+a}$
$\frac{+a}{+0}, \frac{-a}{-0}, \frac{+\infty}{+0}, \frac{-\infty}{-0}$	$\frac{+a}{-0}, \frac{-a}{+0}, \frac{+\infty}{-0}, \frac{-\infty}{+0}$
$\sqrt{+\infty}$	

[1] $\geq 2^{127}(2-2^{-24})$ for single precision, and $\geq 2^{1023}(2-2^{-53})$ for double precision.

[2] $\leq -2^{127}(2-2^{-24})$ for single precision, and $\leq -2^{1023}(2-2^{-53})$ for double precision.

Finite result when one operand is infinite

Cases when result=+0	**Cases when result=−0**
$\frac{+a}{+\infty}, \frac{-a}{-\infty}$	$\frac{+a}{-\infty}, \frac{-a}{+\infty}$
$\frac{+0}{+\infty}, \frac{-0}{-\infty}$	$\frac{+0}{-\infty}, \frac{-0}{+\infty}$

When is the result NaN?

either operand is NaN
$+\infty + -\infty$
$+\infty - +\infty, -\infty - -\infty$
$0 * \infty$
$\frac{\infty}{\infty}$
$\sqrt{-a}, \sqrt{-\infty}$

By the way, IEEE 754 stipulates that $\sqrt{-0} = -0$, $\sqrt{+0} = +0$.

Comparing +0 and −0

We consider the following 5 floating-point comparison operators: $<, ==, >, <=, >=$.
In these operators, +0 and −0 are treated as the same number. That means the following relations are true:

$$+0 == -0, \quad -0 == +0$$

$$+0 <= -0, \quad -0 <= +0$$

$$+0 >= -0, \quad -0 >= +0$$

And the rest of the relations are false.

Comparisons involving NaN

If NaN is involved in any of the 5 comparison operators: <, ==, >, <=, >=. Then the result is **always false**.

That implies, if NaN is involved in the operator ! =, then the result is **always true**. Even when the two numbers compared have exactly the same binary representation!

$$NaN \quad != \quad everything$$
$$NaN \quad != \quad NaN$$

NaN, and only NaN, has this property. So we can test if $x! = x$ to find if x is a NaN.

Error Analysis

Sources of error

Before entering into the numerical process, there are modeling errors, observation errors, etc. They do not belong to this course and will not be discussed further. In numerical analysis, there are two major sources of error:

Rounding error **Rounding error** (or **round-off error**) is the error caused by rounding an arbitrary real number to a machine representable floating-point number. When repeated arithmetic operations are performed, round-off error may occur in each operation and these errors add up.

Truncation error **Truncation error** arises while an infinite series is to be added up to arrive at the exact result, but we are forced to truncate the process after attaining an acceptable level of accuracy due to practicality issues. Most frequently the series is a **Taylor series**. In addition, quite often the exact result is obtained by an infinite iteration, but we are forced to stop the iteration due to time limitation. The error caused in this way is also called a truncation error.

Errors: absolute and relative

Let α be the exact value, and let β be its approximation. Then **error**$= \beta - \alpha$, **absolute error**$= |\beta - \alpha|$, **relative error**$= |\frac{\beta-\alpha}{\alpha}|$.

Notice that in computing the absolute error, the roles of α and β are the same, whereas in computing the relative error, it is essential to distinguish one of the two numbers as correct. (Observe that the relative error is undefined in the case $\alpha = 0$.) For practical reasons, the relative error is usually more meaningful than the absolute error. In summary,

$$\begin{aligned} \text{absolute error} &= |\text{approximate value} - \text{exact value}| \\ \text{relative error} &= |\frac{\text{approximate value} - \text{exact value}}{\text{exact value}}| \end{aligned}$$

Machine epsilon

Definition (machine epsilon)

For a floating-point system, the machine epsilon is the smallest positive number ε such that $1 + \varepsilon$ is representable in the system.

Obviously machine epsilon depends on the floating-point system used. It is clear that for IEEE 754 single precision, the machine epsilon $\varepsilon = 2^{-23}$, since the representable number right next to 1 is $1 + 2^{-23}$. Obviously, ε is itself representable.

Machine epsilon is closely related to the error analysis of basic operations, as we will later see. It is also widely used in numerical programs to formulate accuracy conditions for terminating iterations.

Error analysis for basic operations

Nearly all numerical computations are based on the 5 basic operations: $+, -, *, /, \sqrt{\ }$. Because of their importance, they are implemented by the CPU hardware, complying with the IEEE 754 standard. The error analysis of the basic operations is also very important.

In this section, we will carry out two analyses for each of the 5 operations: output error analysis and input error analysis.

Output error analysis for basic operations

According to the IEEE 754 standard, except for the rare cases of overflow and underflow, the actually implemented basic operations are the exact basic operations followed by rounding. Let $\oplus, \ominus, \otimes, \oslash, \checkmark$ denote the actually implemented $+, -, *, /, \sqrt{\ }$ operations, respectively. Then

$$\begin{aligned}
\oplus(x, y) &= \mathrm{fl}(x + y) \\
\ominus(x, y) &= \mathrm{fl}(x - y) \\
\otimes(x, y) &= \mathrm{fl}(x * y) \\
\oslash(x, y) &= \mathrm{fl}(x/y) \\
\checkmark(x) &= \mathrm{fl}(\sqrt{x})
\end{aligned}$$

where $\mathrm{fl}(x)$ is the machine representable number obtained by rounding x using the roundTiesToEven method.

In output error analysis, we neglect the possible error of the operands. Thus, it is clear that the output error is identical to the rounding error, which we will discuss in the next slide.

Error analysis of rounding

Theorem (relative error bound of rounding)

Let x be a real number and $\mathrm{fl}(x)$ be its machine representation using roundTiesToEven. Suppose x falls within the normalized range, that is, $2^{-126} \leq |x| < 2^{127}(2-2^{-24})$ for single precision or $2^{-1022} \leq |x| < 2^{1023}(2-2^{-53})$ for double precision. Then

$$\left|\frac{\mathrm{fl}(x)-x}{x}\right| < \frac{\varepsilon}{2}$$

where ε is the machine epsilon, which is 2^{-23} for single precision or 2^{-52} for double precision.

This gives an upper bound for the relative errors in rounding, which is 2^{-24} for single precision or 2^{-53} for double precision.

Proof.

Assume, without loss of generality, that $x \in [1, 2)$. Then $\mathrm{fl}(x) \in [1, 2]$, and $\mathrm{fl}(x) = 1 + k\varepsilon$ for some $k = 0, 1, \ldots, \frac{1}{\varepsilon}$. Since roundTiesToEven chooses the nearest, we have

$$|\mathrm{fl}(x) - x| \leq \frac{\varepsilon}{2}.$$

If $x = 1$, then $\mathrm{fl}(x) - x = 0$, the result is obviously true. Otherwise $x > 1$, and we also have

$$\left|\frac{\mathrm{fl}(x) - x}{x}\right| < \frac{\varepsilon}{2}.$$

$\square$

We have several remarks.

- In the normalized range, the biggest relative error is attained when $x = 2^e(1+\frac{\varepsilon}{2})$ where e is any integer exponent in the normalized range. Then $\mathrm{fl}(x) = 2^e$, and

$$\left|\frac{\mathrm{fl}(x) - x}{x}\right| = \frac{\varepsilon}{2+\varepsilon}.$$

- The bound is not applicable to denormalized numbers. For example, in the extreme case, let $x > 0$ be smaller than half the smallest positive representable number. Then $\mathrm{fl}(x) = 0$, and the relative error is 1!
- There is no upper bound on the absolute errors of rounding. And since rounding is so ubiquitous in numerical computing, we can see that the relative error is much more meaningful than the absolute error.

Input error analysis of operations

In input error analysis, we consider that the operands may be contaminated by errors, while neglecting the error in the output side. Let x and y be the ideal operands, and $\tilde{x}$ and $\tilde{y}$ be the actual operands fed into the operation. Let $\circ$ denote the particular operation we are studying. Then the ideal result is $x \circ y$, but the actual result is $\tilde{x} \circ \tilde{y}$.

As we just analyzed, it is more helpful to assume there are small relative errors in the operands. That is,

$$\begin{aligned} \tilde{x} &= x(1+\delta_1) \\ \tilde{y} &= y(1+\delta_2) \end{aligned}$$

where δ_1, δ_2 are small numbers.

In the following slides, we will analyze the relative error of the result caused by input errors, for operations $+, -, *, /, \sqrt{\ }$.

Input error analysis of addition

$$
\begin{aligned}
\text{ideal result} &= x + y \\
\text{actual result} &= x(1+\delta_1) + y(1+\delta_2) \\
\text{absolute error} &= |x\delta_1 + y\delta_2| \\
\text{relative error} &= \left|\frac{x\delta_1 + y\delta_2}{x+y}\right|
\end{aligned}
$$

We can see that if $x + y$ is a small number η, then the relative error is $|\frac{x(\delta_1-\delta_2)}{\eta} + \delta_2|$, which can be very large if η is very small. Therefore, we must avoid situations that the sum of the operands is near zero and at least one of the operands are contaminated by error.

Input error analysis of subtraction

$$
\begin{aligned}
\text{ideal result} &= x - y \\
\text{actual result} &= x(1+\delta_1) - y(1+\delta_2) \\
\text{absolute error} &= |x\delta_1 - y\delta_2| \\
\text{relative error} &= \left|\frac{x\delta_1 - y\delta_2}{x-y}\right|
\end{aligned}
$$

We can see that if $x - y$ is a small number η, then the relative error is $|\delta_1 + \frac{y(\delta_1-\delta_2)}{\eta}|$, which can be very large if η is very small, even leading to the result being meaningless. Therefore, we must avoid situations that the operands are close to one another and at least one of the operands are contaminated by error.

Input error analysis of multiplication

$$
\begin{aligned}
\text{ideal result} &= x * y \\
\text{actual result} &= x(1+\delta_1) * y(1+\delta_2) \\
\text{absolute error} &= |xy(\delta_1 + \delta_2 + \delta_1\delta_2)| \\
\text{relative error} &= |\delta_1 + \delta_2 + \delta_1\delta_2|
\end{aligned}
$$

It can be inferred that in this case there is relative error bound regardless of the operands. So unlike addition and subtraction, there is no precaution against using multiplication in special cases.

Input error analysis of division

$$
\begin{aligned}
\text{ideal result} &= x/y \\
\text{actual result} &= \frac{x(1+\delta_1)}{y(1+\delta_2)} \\
\text{absolute error} &= \left|\frac{x(\delta_1-\delta_2)}{y(1+\delta_2)}\right| \\
\text{relative error} &= \left|\frac{\delta_1-\delta_2}{1+\delta_2}\right|
\end{aligned}
$$

It can be inferred that in this case there is relative error bound regardless of the operands. So there is also no precaution against using division in special cases.

Input error analysis of square root

$$
\begin{aligned}
\text{ideal result} &= \sqrt{x} \\
\text{actual result} &= \sqrt{x(1+\delta)} \\
\text{absolute error} &= |\sqrt{x(1+\delta)} - \sqrt{x}| \\
\text{relative error} &= |\sqrt{1+\delta} - 1|
\end{aligned}
$$

It can be inferred that in this case there is relative error bound regardless of the operands. So again there is no precaution against using square root in special cases.

Some error-avoiding techniques

To reduce error in numerical computing, we note the following as helpful:

- **Use as fewer operations as possible**. We already see that every operation introduces error. Therefore a general principle is to use as fewer operations as possible. This not only helps reduce error, but also improves efficiency.
- **Avoid $x - y$ where x and y are close**. The input error analysis of addition/subtraction has revealed that directly performing $x - y$ when x and y are close is numerically unsound, unless both x and y are error-free, or $x - y$ is itself a target of computation and there is no better way to compute it.
- **Appropriately arrange the order of addition when adding a large set of numbers**. A better approach is to add all the positive numbers and then add all the negative numbers, and finally add the two sums. When adding numbers of same sign, try to first add numbers with small magnitudes.

We will give examples to illustrate the first two points.

Nested multiplication for evaluating polynomials

Example

How do you compute the polynomial $P(x) = c_0 + c_1 x + c_2 x^2 + c_3 x^3 + c_4 x^4$?

- A naive approach is to write $P(x) = c0 + c1 * x + c2 * x * x + c3 * x * x * x + c4 * x * x * x * x$. This uses 10 multiplications and 4 additions.
- Some students will write $P(x) = c0 + c1 * x + c2 * x * x + c3 * pow(x, 3) + c4 * pow(x, 4)$. This is even worse than the first approach, since for small integer n, $pow(x, n)$ is slower and less accurate than $\underbrace{x * \cdots * x}_{n \text{ times}}$.
- The optimal approach is to write $P(x) = ((((c4 * x) + c3) * x + c2) * x + c1) * x + c0$. This uses only 4 multiplications and 4 additions. It is called **nested multiplication** or **Horner's scheme** for evaluating polynomials.

Another example

Example

Computing by formula $\ln x - \ln y$ is numerically unsound, why?

Because we have a significantly better way to compute it, use $\ln(\frac{x}{y})$ instead. The original formula uses 2 ln evaluations and 1 subtraction, while the proposed formula uses 1 ln evaluation and 1 division. Moreover, since the proposed formula does not use subtraction, it is free of the relative-error problem of subtraction.

Still another example

Example

How do you compute $\sqrt{x^2+1}-x$ when x is large?

When x is large, $\sqrt{x^2+1}$ is close to x, therefore the original formula has the relative-error problem of subtraction, since the computation of $\sqrt{x^2+1}$ is not error-free. Fortunately, we have

$$\sqrt{x^2+1}-x=\frac{1}{\sqrt{x^2+1}+x},$$

and we can use $\frac{1}{\sqrt{x^2+1}+x}$ instead. The proposed formula needs one extra division, but the error introduced by the division is much less than the very big relative error from the original subtraction.

Taylor series

Familiar (and useful) examples of Taylor series are the following:

$$
\begin{aligned}
e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{k=0}^{\infty} \frac{x^k}{k!} \\
\sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1} \\
\cos x &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} x^{2k} \\
\frac{1}{1-x} &= 1 + x + x^2 + x^3 + \cdots = \sum_{k=0}^{\infty} x^k \quad (|x| < 1) \\
\ln x &= (x-1) - \frac{(x-1)^2}{2} + \frac{(x-1)^3}{3} - \cdots = \sum_{k=1}^{\infty} \frac{(-1)^{k-1}}{k} (x-1)^k \quad (0 < x \leq 2)
\end{aligned}
$$

Taylor's formula

Closely related to Taylor series is Taylor's formula. It is a representation of a function $f(x)$ as a sum of its Taylor polynomial of degree n $(n = 0, 1, 2, \cdots)$ $P_n(x)$ and a remainder term $r_n(x)$:

$$f(x) = P_n(x) + r_n(x).$$

Generally there are two forms of Taylor's formula:

- Taylor's formula with **Peano remainder**
- Taylor's formula with **Lagrange remainder**

Taylor's formula with Peano remainder

Theorem (Taylor's formula with Peano remainder)

If f is a function such that its nth derivative at a (i.e. $f^{(n)}(a)$) exists then

$$f(a+h) = \underbrace{f(a) + f'(a)h + \frac{f''(a)}{2!}h^2 + \cdots + \frac{f^{(n)}(a)}{n!}h^n}_{\text{Taylor polynomial}} + \underbrace{o(h^n)}_{\text{remainder}}$$

where $o(h^n)$ represents a function $r(h)$ with $\frac{r(h)}{h^n} \to 0$ as $h \to 0$.

It is apparent that Peano's form of remainder gives little information about the remainder. Therefore this form of Taylor's formula is almost useless in numerical analysis. Later we will present Lagrange's form of Taylor's formula. Before that, we need Cauchy's mean value theorem.

Cauchy's mean value theorem

Theorem (Cauchy's mean value)

Let functions f and g be continuous on an interval $[a, b]$ differentiable on (a, b), and $g'(x) \neq 0$ for all $x \in (a, b)$. Then there is $c \in (a, b)$ such that

$$\frac{f(b) - f(a)}{g(b) - g(a)} = \frac{f'(c)}{g'(c)}.$$

Proof.

We have $g(a) \neq g(b)$. Otherwise, by Rolle's theorem there is some d where $g'(d) = 0$, contradicting our assumption. Now let

$$h(x) = f(x) - f(a) - \frac{f(b) - f(a)}{g(b) - g(a)}(g(x) - g(a)).$$

Then h is continuous on $[a, b]$ and differentiable on (a, b) and $h(a) = h(b)$. By Rolle's theorem, there is $c \in (a, b)$ such that $h'(c) = 0$, i.e. $f'(c) - \frac{f(b)-f(a)}{g(b)-g(a)} g'(c) = 0$. Hence the result. □

Taylor's formula with Lagrange remainder

Theorem (Taylor's)

Suppose that function f has continuous nth derivative on the interval $[\alpha, \beta]$, and has $(n+1)$th derivative on (α, β). Let $a \in (\alpha, \beta)$ and $x \in [\alpha, \beta]$. Then there exists ξ lying between a and x such that

$$f(x) = \underbrace{f(a) + \sum_{k=1}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k}_{\text{Taylor polynomial}} + \underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!}(x-a)^{n+1}}_{\text{remainder}}.$$

The remainder is actually the truncation error that we have mentioned before. Therefore Taylor's formula with Lagrange remainder provides a tool for truncation error analysis. We will give a proof of this theorem in the next slide.

Proof of Taylor's theorem.

If $a = x$, then the result is clearly true. Let us assume, without loss of generality, that $a < x$. Define two functions F and G by

$$\begin{aligned} F(t) &= f(t) + \sum_{k=1}^{n} \frac{f^{(k)}(t)}{k!}(x-t)^k \\ G(t) &= (x-t)^{n+1}. \end{aligned}$$

Then F and G are continuous on $[a, x]$ and differentiable on (a, x). Then for $t \in (a, x)$, $F'(t) = f'(t) - \sum_{k=1}^{n} \frac{f^{(k)}(t)}{(k-1)!}(x-t)^{k-1} + \underbrace{\sum_{k=1}^{n} \frac{f^{(k+1)}(t)}{k!}(x-t)^k}_{=\sum_{k=2}^{n+1} \frac{f^{k}(t)}{(k-1)!}(x-t)^{k-1}} = \frac{f^{(n+1)}(t)}{n!}(x-t)^n$, and $G'(t) = -(n+1)(x-t)^n$. By Cauchy's mean value theorem, there is $\xi \in (a, x)$ such that $\frac{F(x)-F(a)}{G(x)-G(a)} = \frac{F'(\xi)}{G'(\xi)}$. That is,

$$\frac{f(x) - [f(a) + \sum_{k=1}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k]}{-(x-a)^{n+1}} = \frac{\frac{f^{(n+1)}(\xi)}{n!}(x-\xi)^n}{-(n+1)(x-\xi)^n} = -\frac{f^{(n+1)}(\xi)}{(n+1)!}.$$

Hence the result. □

Using Taylor's formula

Example

Expand $\sqrt{1+h}$ in powers of h. Then compute $\sqrt{1.00001}$ and $\sqrt{0.99999}$.

Using Taylor's formula

Example

Expand $\sqrt{1+h}$ in powers of h. Then compute $\sqrt{1.00001}$ and $\sqrt{0.99999}$.

Let $f(x) = \sqrt{x}$. Then $f'(x) = \frac{1}{2}x^{-1/2}$, $f''(x) = -\frac{1}{4}x^{-3/2}$, $f'''(x) = \frac{3}{8}x^{-5/2}$, and so on. Now use Taylor's formula with $x = 1 + h$. Taking $n = 2$ for illustration, we have

$$\sqrt{1+h} = 1 + \frac{1}{2}h - \frac{1}{8}h^2 + \frac{1}{16}h^3\xi^{-5/2}$$

where ξ is an unknown number that satisfies $1 < \xi < 1 + h$. Then

$$\sqrt{1.00001} \approx 1 + \frac{1}{2} * 10^{-5} - \frac{1}{8} * 10^{-10} = 1.000004999987500,$$

$$\sqrt{0.99999} \approx 1 - \frac{1}{2} * 10^{-5} - \frac{1}{8} * 10^{-10} = 0.999994999987500.$$

Since $1 < \xi < 1 + h$, the absolute error does not exceed

$$\frac{1}{16}h^3\xi^{-5/2} < \frac{1}{16}10^{-15} = 0.0000000000000000625$$

and both numerical values are correct to all 15 decimal places shown.